PROITBRIDGE

Strive For Better Future

System Design for Generative AI

10 Engineering Principles Every GenAI Builder Should Know

Gateway | Retrieval | Memory | Guardrails | Routing | Observability

Gateway · Retrieval · Memory · Guardrails · Routing · Observability

BUILT WITH BEST-IN-CLASS TECHNOLOGIES

OpenAI | ANTHROPIC | Meta | LangChain | Pinecone

Weaviate | PostgreSQL | Redis | Docker | Kubernetes

Design Smarter. Build Robustly.
Scale Confidently. Improve Continuously.

SWIPE TO LEARN AND BUILD

Practical. Scalable.
Production-Ready.

THE FOUNDATION LAYER

Where Every Request Begins — Gateway & Prompt Orchestration



The Core Insight: Before the LLM ever sees a single token, two critical layers decide whether your system is secure, stable, and intelligent. Skip either one and your system breaks under real users.

API Gateway

What it is:

The traffic controller that sits between your users and everything else.

What it handles:

Incoming Request
(1000 users at once)

API GATEWAY

- **Authentication** — Is this user allowed in?
- **Rate Limiting** — Max 60 req/min per user
- **API Key Guard** — Valid key? Proceed.
- **Request Routing** — Which service handles this?
- **Load Balancing** — Distribute across nodes

Routed safely to the right service

Real Example

1000 users hit your AI chatbot simultaneously. Without a gateway, all 1000 requests slam directly into your LLM API — you hit rate limits, costs spike, some users get errors, the whole system degrades. The gateway queues, throttles, and routes them gracefully so every user gets a response.

Without this:

System overload → cascading failures → massive cost spike → angry users.

Prompt Orchestration

What it is:

Not just "write a prompt." Prompt orchestration is the logic layer that dynamically assembles the right prompt for the right context at runtime.

User Query:
"How do I reset my password?"

PROMPT ORCHESTRATION

- **System Prompt** — "You are a helpful customer support agent for Acme Inc."
- **Role Assignment** — Support specialist, polite, concise tone
- **Context Inject** — Retrieved policy docs + user account info
- **Few-Shot Examples** — 2 example Q&A pairs for format guidance
- **Dynamic Fill** — User name, tier, conversation history

Final assembled prompt sent to LLM

Real Example

A healthcare assistant and a coding assistant use completely different system prompts, tones, safety instructions, and output formats — even if they run on the same model.

Key rule:

Version your prompts like code. Every prompt change is a deployment. Track, test, and roll back when needed.

Foundation Layer Summary

API Gateway
Who gets in, how fast, to where

Prompt Orchestration
What the LLM actually sees and how

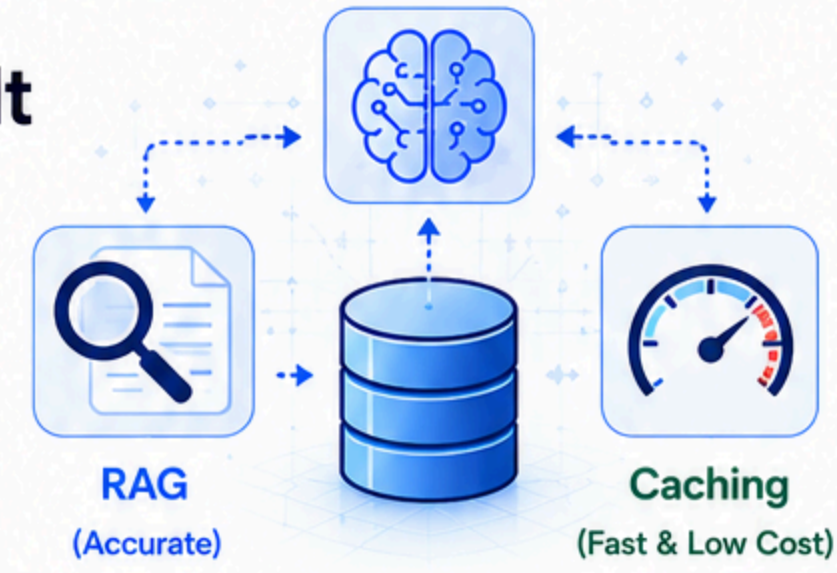
PROITBRIDGE
Strive For Better Future



AI Coach John
Follow for more
Learn. Build. Ship. Repeat.

THE KNOWLEDGE LAYER

What the LLM Knows vs What It Needs to Know — RAG & Caching



The Core Insight: LLMs are frozen at their training cutoff. They know nothing about your product, your policies, your customers, or anything that happened last week. RAG fixes the knowledge gap. Caching fixes the cost and speed gap.

RAG / Knowledge Retrieval

What it is:

Retrieval-Augmented Generation. Instead of relying on what the model memorized during training, you retrieve relevant information at query time and inject it into the prompt as context.

Full RAG Flow:

INDEXING PHASE (runs once, offline)

Your Documents (PDFs, Docs, DBs, HTML)

↓

Chunking Layer
(split into 300–512 token pieces with overlap)

↓

Embedding Model
(convert each chunk → dense vector)

↓

Vector Database
(store all vectors + metadata)
[Pinecone / pgvector / Weaviate / Qdrant]

QUERY PHASE (runs every request, online)

User Query: "What is our refund policy?"

↓

Embed the query
(same embedding model as indexing)

↓

Vector Similarity Search
(find Top-K most similar chunks)

↓

Reranker (optional but recommended)
(re-score chunks with cross-encoder)

↓

Context Injection
(insert retrieved chunks into prompt)

↓

LLM generates grounded answer

↓

Response cites actual policy document

What RAG is used for:

- Internal knowledge bases and wikis
- Product documentation and support
- Legal and compliance documents
- Company-specific policies and procedures
- Any data that changes faster than model retraining

Real Example:

User asks: "What is our refund policy for enterprise customers?"

Without RAG, the LLM guesses or says it doesn't know. With RAG, it retrieves the exact policy document chunk, reads it, and answers accurately with the correct terms.

Caching

What it is:

A layer that stores previous LLM responses and serves them instantly for identical or similar queries — without making an expensive model call.

Caching Flow:

User sends query

↓

CACHE CHECK

Exact match found?
YES → Return cached response → milliseconds, zero cost
NO ↓

Semantic match found?
(similar meaning, not exact)
YES → Return cached response → still zero LLM cost
NO ↓

↓

Call LLM → get response

↓

Store in cache for next time

↓

Return to user

What you can cache:

Layer	What to cache	TTL
Embedding cache	query vectors	24h+
Prompt cache	identical system prompts	session
Output cache	full LLM responses	1h–24h
RAG cache	retrieved chunks	1h–7d

Real Example:

Your support chatbot receives "How do I reset my password?" 500 times per day. Without caching: 500 LLM API calls, 500× the cost, 500× the latency. With caching: 1 LLM call, 499 instant cache hits. Cost reduction: ~99%.

The numbers:

- Cached response: 50–100ms
- Uncached LLM response: 1000–5000ms
- Cost: $0 for cached vs $0.001–0.02 per LLM call

Knowledge Layer Summary

RAG
→ Connect LLM to your data and make answers accurate

Caching
→ Stop calling the LLM when you already have the answer

PROITBRIDGE
Strive For Better Future



AI Coach John
Follow for more
Learn. Build. Ship. Repeat.

THE MEMORY & SAFETY LAYER

What the System Remembers and What It Must Never Do — Memory & Guardrails



The Core Insight: A stateless AI that forgets every conversation is frustrating. An AI with no safety controls is dangerous. Memory makes the system feel intelligent and personal. Guardrails make it safe to ship to real users.

Memory Management

What it is:

The mechanisms that allow your AI system to remember information across a conversation, a session, or across multiple sessions over time.

Four Types of Memory:

MEMORY TYPES	
IN-CONTEXT (Short-term)	Lives in the current prompt window Conversation turns so far Lost when context window fills up Example: "You said X earlier..."
SESSION (Medium-term)	Persists for duration of one user session (stored in DB/cache) Cleared after session ends Example: Items added to cart
LONG-TERM (Persistent)	Persists across all sessions Stored in vector DB or SQL Retrieved via semantic search Example: User preferences, history
ENTITY (Structured)	Tracks specific named entities Names, roles, preferences, facts Updated as new info arrives Example: "Alex prefers Python"

Memory Flow in Practice:

- Session Start
- Load long-term memory from DB (What do we know about this user?)
- Build in-context memory (Inject relevant past facts into system prompt)
- Conversation Runs...
- Short-term memory builds up (conversation turns)
- Entity extractor identifies new facts "My name is Alex, I'm a Python developer" → Store: {name: Alex, language: Python}
- Session Ends
- Persist updated memory to long-term store

Real Example:

Turn 1: User says "My name is Alex and I'm learning ML."
Turn 5: User asks "Recommend a course for me."
Without memory: "Could you tell me your background?" (frustrating)
With memory: "Based on your ML learning journey, I'd recommend...".

Safety & Guardrails

What it is:

A mandatory protection layer that runs on every input before the LLM sees it, and every output before the user sees it. Not optional. Not an afterthought.

What you're protecting against:

THREAT		WHAT HAPPENS
Prompt Injection	→	User hijacks system prompt behavior
Jailbreaking	→	User bypasses safety training
Data Leakage	→	System prompt or context exposed
Harmful Outputs	→	Model generates dangerous content
PII Exposure	→	Personal data leaked in responses
Hallucination	→	False facts delivered as truth

The Guardrails Architecture:

User Input

INPUT GUARDRAILS
- Injection detector
- Toxicity classifier
- PII detector + redactor
- Topic scope filter
- Input length limiter

LLM CALL

OUTPUT GUARDRAILS
- Content safety classifier
- PII scrubber
- Hallucination detector
- Format validator
- Sensitive topic filter

User sees response

Guardrail Tools:

- **LlamaGuard** → Meta's input + output safety classifier
- **NeMo Guardrails** → NVIDIA's programmable guardrail framework
- **Guardrails AI** → Schema + content validation
- **Azure Content Safety** → Microsoft moderation API
- **OpenAI Moderation** → Free content classification endpoint

The rule: Safety is architecture. Every production system needs guardrails before the first real user arrives.

Memory & Safety Layer Summary

Memory → Make the system feel intelligent and personal

Guardrails → Make the system safe enough to ship to anyone

PROITBRIDGE
Strive For Better Future



AI Coach John
Follow for more
Learn. Build. Ship. Repeat.

THE INTELLIGENCE LAYER

Smarter Routing, Better Visibility — Model Selection & Observability



The Core Insight: Not every task needs your best and most expensive model. Smart routing directs each request to the right model for the job — saving cost without sacrificing quality. Observability makes sure you can see exactly what's happening when things go wrong, because they will.

Model Selection & Routing

What it is:

A routing layer that analyzes each incoming request and directs it to the most appropriate model based on task complexity, cost requirements, latency needs, and capability requirements.

Why one model isn't enough:

Model Type	Speed	Cost	Best For
Small (7B–13B)	Very Fast	Very Low	Classification, simple QA, routing itself
Medium (70B)	Fast	Medium	Summarization, extraction, chat
Large (GPT-4o, Claude)	Slower	High	Reasoning, analysis, complex generation
Specialized	Varies	Varies	Code, math, vision, multilingual

Model Routing Flow:

Incoming Request

↓

ROUTER / CLASSIFIER

Analyze request:
- How complex is this task?
- Does it need reasoning?
- Is it latency sensitive?
- What's the cost budget for this?

Small LLM (fast, cheap)	Medium LLM (balanced)	Large LLM (powerful, expensive)
"Is this spam?"	"Summarize this doc"	"Analyze strategy"

Real-World Routing Rules:

Task	→	Model Choice
Intent classification	→	Small fast model
Simple FAQ answer	→	Small fast model
Email summarization	→	Medium model
Customer support reply	→	Medium model
Complex code generation	→	Large model (GPT-4o)
Multi-step reasoning task	→	Large model (Claude)
Image understanding	→	Vision model
Real-time voice response	→	Fastest available model

Cost Impact:
Routing 60% of requests to a small model and 40% to a large model can reduce LLM costs by 50–70% with minimal quallity impact on the tasks the small model handles well.

Monitoring & Observability

What it is:

The full visibility layer that tells you what your AI system is doing, how well it's doing it, where it's failing, and what it costs — in real time and historically.

What to track:

PERFORMANCE METRICS

Time to First Token (TTFT)	→	Perceived speed
Tokens per Second (TPS)	→	Generation throughput
P50 / P95 / P99 Latency	→	Tail latency (catches slow outliers)
Error Rate	→	% of failed or malformed calls

COST METRICS

Input tokens per request	→	Prompt cost
Output tokens per request	→	Generation cost
Cost per query	→	Full request economics
Cache hit rate	→	% of requests served from cache
Daily/Monthly spend	→	Budget tracking + anomaly alerts

QUALITY METRICS

Hallucination rate	→	Via automated eval pipeline
Guardrail trigger rate	→	How often safety filters fire
User satisfaction score	→	Thumbs up/down, ratings
Retrieval precision	→	RAG: were the right chunks found?
Answer faithfulness	→	RAG: did answer use the context?

The Observability Stack:

User Request

↓

Trace ID assigned (follows request end-to-end)

Span 1: Gateway	latency: 5ms
Span 2: RAG retrieval	latency: 120ms, chunks: 3
Span 3: Prompt assembly	latency: 10ms, tokens: 2847
Span 4: LLM call	latency: 1850ms, output: 312 tokens
Span 5: Output guardrail	latency: 45ms, passed: true
Span 6: Response delivery	latency: 8ms

Total: 2038ms | Cost: $0.0043 | Status: success

Tools:

LangSmith	→	LangChain-native tracing + eval
Langfuse	→	Open-source, any framework
Helicone	→	Cost tracking + caching layer
Arize Phoenix	→	LLM + embedding observability
Datadog	→	Enterprise-grade metrics + alerting

Key questions observability answers:

- "Why did that output fail?" → Trace the exact request
- "Why did cost spike this morning?" → Token usage breakdown
- "Where is latency coming from?" → Per-span timing
- "Which users are hitting guardrails?" → Safety trigger log
- "Is retrieval quality degrading?" → RAG eval metrics over time

The rule:
Safety is architecture. Every production system needs guardrails before the first real user arrives.

Intelligence Layer Summary

Model Routing → Right model for right task = quality at lower cost

Observability → You cannot fix what you cannot see

PROITBRIDGE
Strive For Better Future



AI Coach John
Follow for more
Learn. Build. Ship. Repeat.

THE IMPROVEMENT LAYER & FULL ARCHITECTURE

Scale It. Improve It. Ship It. — Load Balancing, Feedback & The Full Picture



The Core Insight: A system that works for 10 users and a system that works for 100,000 users are architecturally different. And a GenAI system that doesn't continuously improve is falling behind. The final layer covers both: scaling to handle traffic and iterating to handle quality.

9 Load Balancing & Scaling

What it is:

The infrastructure layer that ensures your system handles traffic spikes, distributes load across resources, and processes requests without bottlenecks — whether it's 10 users or 100,000.

The Scaling Challenge:

Demo environment:
10 users → 1 server → LLM API → fine

Production reality:
100,000 users → peak hour traffic → everything breaks if you didn't architect for it

Scaling Architecture:

- 100,000 Users
- Load Balancer (distribute across instances)
  - App
  - App
  - App
  - ← horizontal scaling — add more nodes
- Message Queue (Kafka / SQS / RabbitMQ) ← absorbs traffic bursts
- Async Workers ← process at controlled rate
- LLM API Pool ← multiple keys, multiple providers
- Response Cache ← serve repeated answers instantly
- User gets response

Key Scaling Concepts:

Horizontal Scaling:
Add more application instances instead of bigger machines. Stateless services scale easily. Keep LLM call logic stateless.

Message Queues:
When traffic spikes, requests queue instead of dropping. Workers pull from the queue at a controlled rate. Prevents overloading the LLM API.

Async Processing:
For non-real-time tasks (report generation, batch summarization), process in the background. Return a job ID immediately, deliver result when ready.

Multi-Provider Routing:
Route across OpenAI, Anthropic, and open-source models. If one provider is slow or down, route to another automatically. Prevents single-provider lock-in failures.

10 Evaluation & Improvement Loop

What it is:

The continuous feedback cycle that turns production data into system improvements. Every user interaction is a signal. Every failure is a lesson. The improvement loop turns both into better prompts, better retrieval, and better models.

The Feedback Loop:

User Interacts with System

SIGNAL COLLECTION
- Thumbs up / thumbs down
- Follow-up questions
- User corrections
- Abandonment (no follow-up)
- Time spent reading response

AUTOMATED EVAL
- Run LLM-as-judge on flagged outputs
- Check RAG faithfulness scores
- Detect hallucinations
- Measure format compliance

HUMAN REVIEW QUEUE
- Low-confidence outputs
- Guardrail-triggered responses
- User-flagged responses
- Safety edge cases

IMPROVEMENT ACTIONS
- Update prompts (versioned deploy)
- Fix retrieval chunks
- Add new few-shot examples
- Fine-tune on failure cases
- Adjust guardrail thresholds

Better System ↓ (loop repeats forever)

What improves through iteration:

Prompts → Better instructions, better examples, tighter format

Retrieval → Better chunking, better reranking, better metadata

Models → Fine-tune on high-quality production examples

Guardrails → Tune thresholds based on real false positive rates

Routing → Calibrate which tasks actually need large models

Tools:

- **LangSmith** → LangChain-native tracing + eval
- **Langfuse** → Open-source, any framework
- **Helicone** → Cost tracking + caching layer
- **Arize Phoenix** → LLM + embedding observability
- **Datadog** → Enterprise-grade metrics + alerting

The Complete GenAI System Architecture:

USER → API GATEWAY (Auth, Rate Limit, Route) → PROMPT ORCHESTRATION (Templates, Context, Roles) → RAG (Retrieval) / MEMORY (Management) / CACHE (Layer) → MODEL ROUTER (Small, Medium, Large) → LLM (GPT-4o, Claude, LLaMA) → GUARDRAILS (Safety, PII, Injection) → OBSERVABILITY (Trace, Cost, Quality) → FEEDBACK LOOP (Signals → Improvements) → USER

Continuous Improvement Cycle

Intelligence Layer Summary

Model Routing
→ Right model for right task = quality at lower cost

Observability
→ You cannot fix what you cannot see

PROITBRIDGE
Strive For Better Future



AI Coach John
Follow for more
Learn. Build. Ship. Repeat.